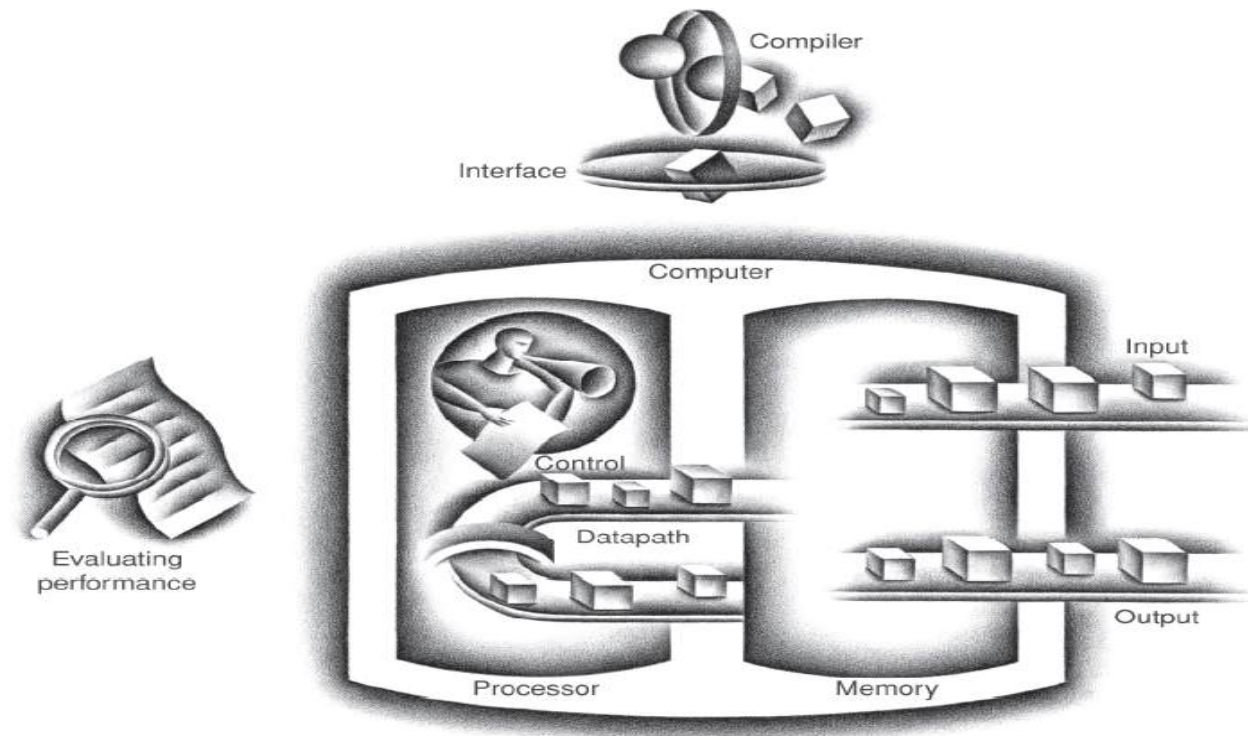


컴퓨터 구조 & 시스템 프로그래밍



2020/12/22 ~

목차

- 기초 개념
- Compiler , Assembler , Interpreter
- 1강_컴퓨터 추상화 및 관련 기술
- 2강_명령어 : 컴퓨터 언어
- 3강_컴퓨터 연산
- 5강_메모리 계층 구조

<기초 개념>

1. 하드웨어 vs 소프트웨어

1) 하드웨어

= CPU , RAM , 메인보드 , etc

2) 소프트웨어

= '프로그램 + 데이터'가 정확한 정의지만 '소프트웨어 = 프로그램'으로 생각해도 무방합니다.

- 프로그래밍 언어로 구성되어 있습니다.

① 시스템 소프트웨어

= 다른 소프트웨어들이 구동될 수 있는 환경을 만들어주는 소프트웨어 (플랫폼)

- 시스템 소프트웨어 바로 아래에 하드웨어가 존재하며 , 위에는 응용 소프트웨어가 존재합니다.

- 시스템 소프트웨어를 과묵한 아버지(하드웨어에 해당)와 자식들(응용 소프트웨어에 해당) 사이를 이어주는 어머니 같은 소프트웨어라고 이해하면 대충 맞습니다.

ex) OS , 게임 엔진(Unity), 컴파일러 , 어셈블러 , 로더

② 응용 소프트웨어

= 사용자가 원하는 기능을 제공하는 프로그램

- 보통 응용 프로그램이라고 이는 우리가 흔히 아는 Application(앱) 입니다.

ex) 게임 , 인터넷 , 한글 문서 , 엑셀

③ 드라이버

= 마우스 , 키보드 , 모니터 , 헤드셋 등을 본체와 연결시키는 소프트웨어

ex) 로지텍 마우스 드라이버

④ 데이터

= 동작하지 않는 객체

- 하드 디스크에서 가장 많이 차지하고 있는 소프트웨어.

- 개발자는 코딩을 하여 소스 파일을 관리하는 것도 중요하지만 소스 파일과 관련된 데이터 또한 관리해야 하고 데이터가 소스 파일 보다 훨씬 양이 많다.

ex) jpg , mp3 , gif 파일

3) 추상화

= 하드웨어를 쉽게 사용하기 위해 하드웨어를 추상화하여 (시스템) 소프트웨어로 만들어서 사용자가 사용합니다.

- 추상화는 OS와 깊게 연관되어 있습니다.

<Compiler , Assembler , Interpreter> 12/22

1. 용어

1) Low-Level Language

= 컴퓨터가 사용하는 기계어(Machine Code)에 가까운 언어

ex) Assembly Language , Machine Code

2) High-Level Language

= 인간이 사용하는 자연어에 가까운 언어

ex) C , C++ , JAVA , Python

3) Assembly Language (어셈블리 언어)

= High-Level Language 보다 훨씬 기계어에 가깝지만 , 컴퓨터 보다는 인간의 언어와 가까운 저급 언어

- 여전히 컴퓨터는 어셈블리 언어를 해석할 수 없습니다.

- **Assembler**를 이용하여 어셈블리 언어를 **Object Program**으로 만들어 줍니다.

- C언어는 개발자가 작성한 High-Level Language(.C)를 어셈블리어로 변환한 후에 Assembler가 Object Program으로 변환 시킵니다.

sw \$4, 0 (\$29)

addi \$4, \$4, -1

jal fact

lw \$4, 0 (\$29)

mult \$2, \$4

어셈블리어 [편집]

기계어는 실제로 컴퓨터의 CPU가 읽어서 실행할 수 있는 0과 1로 이루어진 명령어의 조합이다. 이러한 각 명령어에 대해 사람이 알아보기 쉬운 **니모닉 기호**(mnemonic symbol)를 정해 사람이 좀 더 쉽게 컴퓨터의 행동을 제어할 수 있도록 한 것이 어셈블리 언어이다.

예를 들어,

```
10110000 01100001
```

는 x86 계열 CPU의 기계어 명령이고, 이것을 어셈블리어로 옮겨쓰면 다음과 같다.

```
mov al, 061h
```

4) Object Program

- = 컴파일러의 번역 OR 어셈블러의 번역의 결과로 생성되는 Code의 집합
 - Object Code들이 모여서 Object Program을 만들기 때문에 굳이 구분할 필요는 없습니다.
 - 기계어에 준하는 Low-Level Language입니다.
 - **Object Program은 16진수로 나타내며 , 이를 2진수인 기계어로 변환하는 일은 매우 간단하기 때문에 두 개념을 같게 볼 수도 있습니다.**
- ex) 17202D 69202D 4B101036 032026

5) Machine Code (기계어)

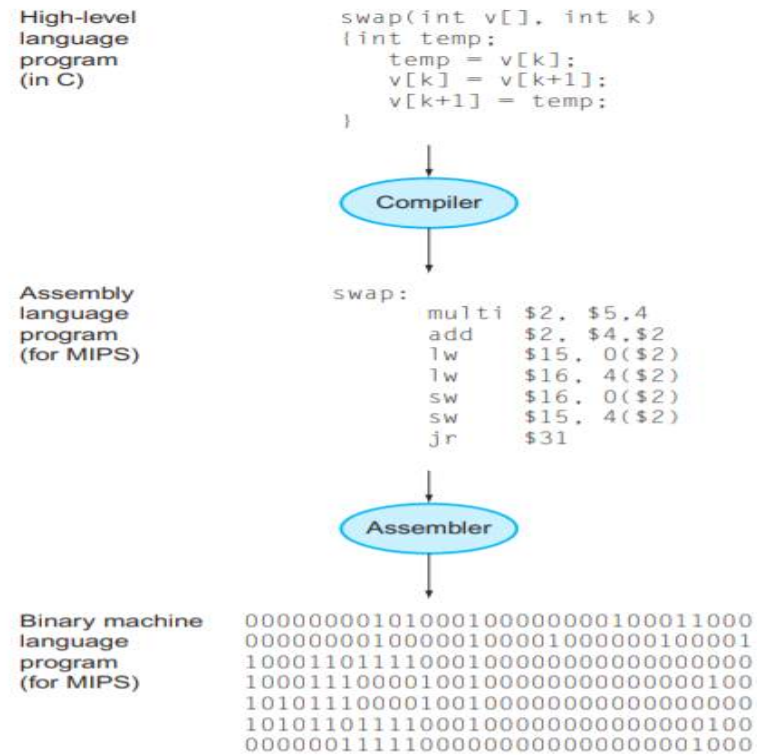
Machine Code

```
10011101000110100000  
01100011010001110110  
10000010111101101110  
11110110001011011000  
10000010011100011011  
10010011000111000000
```

- = CPU가 직접 사용하는 언어
- Binary Code(이진 코드)는 두 개의 값(0과 1)만을 사용하는 코드이므로 기계어와 동일한 의미를 갖습니다.
- 그러므로 **Machine Code = Binary Code**

2. Compiler vs Assembler vs Interpreter

1) Compiler vs Assembler



- 현대에는 **compiler**가 중간과정을 없애고 직접 기계어 프로그램을 생성하기도 합니다.

2) Compiler vs Interpreter

Compiler	Interpreter
번역기	통역기
고급언어	기계언어
짜여진 소스코드 전체를 한번에 번역함	명령 하나하나 실행할 때마다 번역
전체 소스코드 번역에는 많은 시간이 걸리지만 전체 실행 시간은 비교적 빠름	소스코드 한줄 번역에는 적은 시간이 걸리지만 전체 실행 시간은 비교적 느림.
번역 후에 오류를 알려줌. 디버깅 어려움	수정 및 디버깅 쉬움
Java, C, C++ 등을 번역할 때 사용	Python, Perl 등을 번역할 때 사용

<https://saeatechnote.tistory.com/5>

3) VS Code에서 disassemble 하는 방법

- 내가 VS Code 에서 고급 언어로 작성한 소스 파일들이 어셈블 되는 과정을 보면 올바른 학습이 될 것 입니다.
- 디스어셈블을 한 코드가 중요하지는 않습니다. 단지 , 고급언어가 기계어가 되고 , 중간 과정을 실제 코드에서 확인하는 것에 의의를 둡시다.

① g++ -c 소스파일.cpp

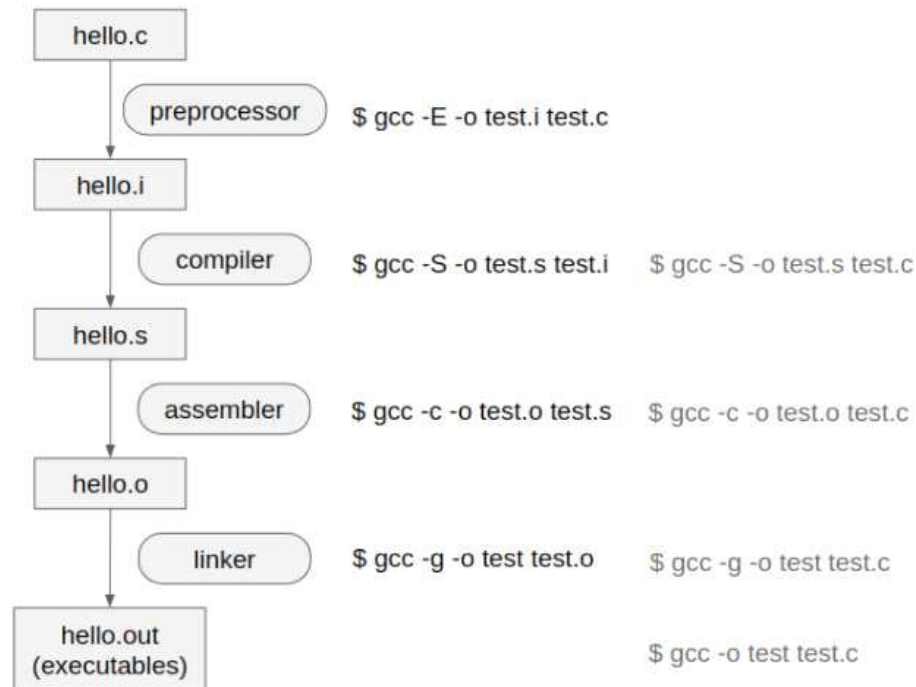
- gcc OR g++의 -c옵션을 사용해서 오브젝트 파일(.o)을 만들어 줍니다.
- 오브젝트 파일은 기계어와 거의 동일합니다.

② disassemble

- 디스어셈블 : 오브젝트 파일을 어셈블리 언어로 변경 하는 과정
- **objdump -d 오브젝트 파일**

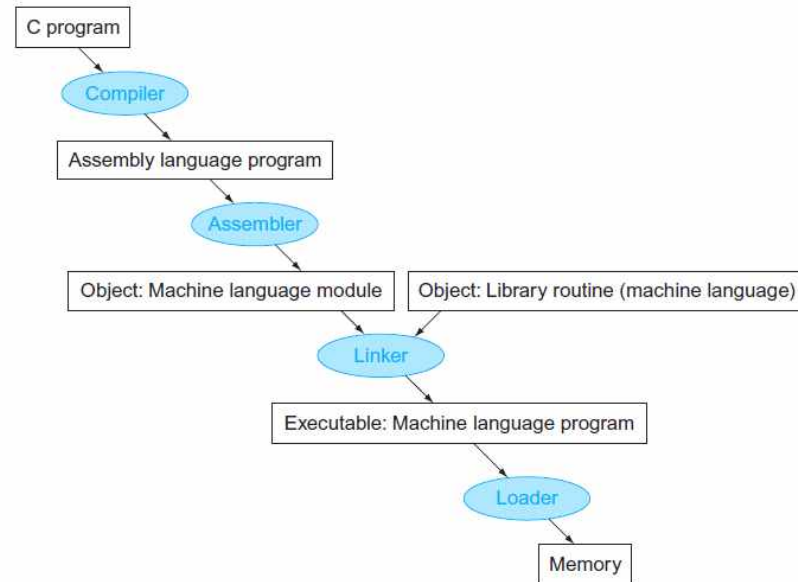
3. C-Language 에서의 번역 과정

1) 자세한 그림 (Loader 제외)



- 고급 언어마다 기계어로의 번역이 다르므로 C-Language에 적용되는 내용입니다.
- C 계열 언어에서 컴파일 과정과 어셈블 과정이 모두 일어납니다.
- **Compiler가 컴파일을 하여 고급 언어 프로그램을 어셈블리 언어 프로그램으로 번역하고 Assembler가 어셈블을 하여 어셈블리 언어 프로그램을 링커 이전의 오브젝트 프로그램(기계어)로 번역합니다.**
- gcc와 g++의 컴파일 옵션은 동일합니다.
- gcc -c : 소스 파일을 compile + assemble 시킵니다.
- gcc -o exe_name test.c : test.c를 exe_name의 이름을 가진 실행 파일로 만들어 줍니다.
- **gcc에 옵션을 붙이지 않으면 compile , assemble , link를 모두 수행합니다.**

2) 간단한 그림 (Loader 포함)



*** OS_study_13강_Address Spaces & Memory Management를 참고하세요 ***

<링커>

- Assemble 과정을 마친 object program은 기계어 이지만 아직 실행 시킬 수 없고 , 프로그램을 시작하는 코드인 Start-Up 코드를 붙여야 비로소 실행파일(.exe)이 됩니다. 이 과정을 링커가 하며 이 과정을 링킹과정이라고 합니다.
- 현대의 프로그램은 여러 가지 작은 프로그램들이 각각 연결(link)되어 하나의 프로그램을 구성합니다.

<로더>

- 링킹 과정까지 완료해서 만들어진 .exe파일은 아직 디스크에 저장되어있습니다. 사용자가 실행 파일을 실행시키면 로더가 디스크에 존재하는 해당 프로그램(.exe파일)을 메모리의 특정 주소에 올립니다(Load).
- 위의 과정은 OS_study에서 자세하게 공부했습니다.

<1강_컴퓨터 추상화 및 관련 기술>

*** 어떻게 해야 IC와 CPI를 줄일 수 있는 지를 고민하는 것이 Computer Architecture입니다. ***

1. CPU Performance and Evaluation

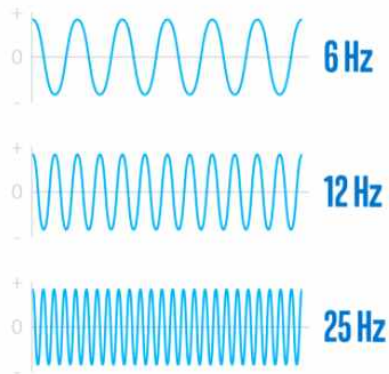
1) Clock Speed

= 1초에 몇 번이나 0과 1의 반복을 통해(사이클) 연산을 하는지에 대한 수치입니다.

- 간단하게 CPU의 속도입니다.

- 주파수의 단위인 Hz로 나타냅니다.

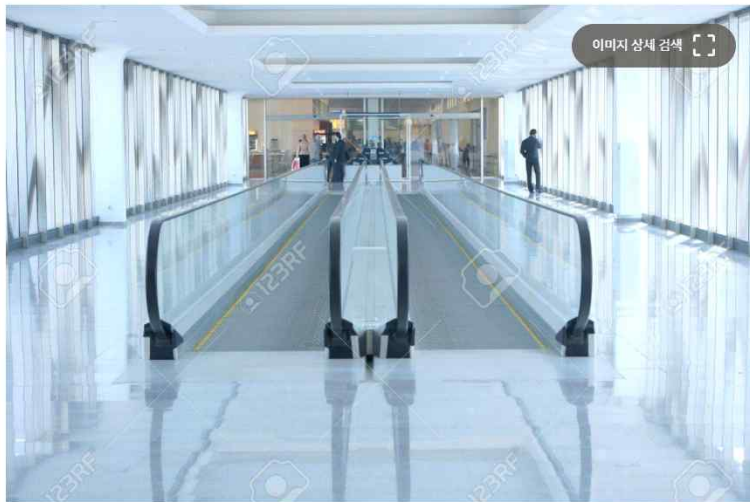
ex) 3.2GHz = 초당 32억 번의 사이클을 수행합니다.



2) Core Count

= CPU가 몇 개의 코어로 분할되어 있는지에 대한 수치입니다.

- 기본적으로 1개의 코어를 갖는 CPU(싱글코어)는 1개의 프로세스만 처리할 수 있습니다.
- 2004년에 기술자들은 CPU의 발열 때문에 더 이상 단일 코어 CPU의 Clock Speed를 높이는 것이 어렵다고 판단하여 코어의 개수를 늘리기 시작했습니다.
- 2007년에 쿼드 코어가 나옵니다.
- 쿼드 코어에 4GHz라고 적힌 CPU는 4GHz의 속도를 가진 코어가 4개로 구성된 CPU입니다.



공항 에스컬레이터

- 1.0GHz 코어가 4개 이면 4.0GHz일 것 같지만 사실은 전혀 다른 이야기 입니다. 이를 좌측의 공항 에스컬레이터로 설명해보겠습니다.
- 4.0GHz 싱글코어 CPU는 초고속 에스컬레이터 1개로 비유할 수 있고 , 1.0GHz코어 쿼드코어 CPU는 저속 에스컬레이터 4개로 비유할 수 있습니다.
- 공항에 사람이 적다면 전자가 빠르겠지만 사람이 많다면 후자가 훨씬 빠릅니다. 현대의 컴퓨터 환경은 사람이 많은 상황이므로 1.0GHz 쿼드 코어가 훨씬 빠르게 느껴집니다,
- 현대 Computer Architecture에서는 더 이상 싱글코어의 클럭 속도를 높이는 데에 한계를 느껴 코어의 개수를 증가시키는 방향으로 발전을 진행하고 있습니다.

- 멀티 코어에 최적화 된 프로그램이 아니라면 멀티 코어에서 제 성능을 내지 못합니다. 하지만 현대에는 스마트폰도 쿼드코어 이므로 코어가 높으면 대부분 좋습니다.

참고 사이트 : <https://winnervps.com/index.php/knowledgebase/198/CPU-Core-Count-vs-Clock-Speed-Which-Is-Better.html>

3) Hyper Threading(HT)

= 원래 CPU에서 하나의 코어는 한 개의 스레드로 한 가지 프로세스만 처리 할 수 있습니다. 하지만 Hyper Threading 기술을 적용하면 **한 개의 코어가 2개의 스레드로 나누어져서 두 가지 작업을 할 수 있게 됩니다.**

- 논리 프로세서는 스레드와 같은 말입니다.
- 2002년에 인텔에서 발표한 기술입니다.

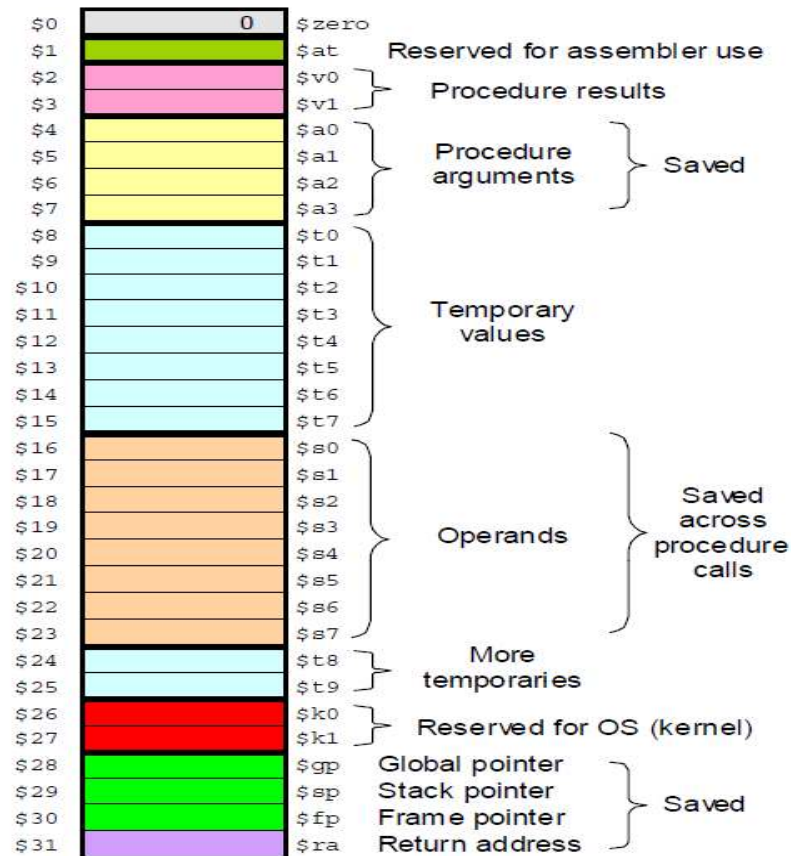
ex) 4 코어 8 스레드 (I7의 초기 특징입니다.)

4) Instruction count(IC) : 프로그램 한 개의 명령어 개수

5) Cycles per Instruction(CPI) : 명령어 한 개당 몇 클락 걸리는지. 알고리즘 , PL , 컴파일러에 따라 달라집니다.

<2강_명령어 : 컴퓨터 언어>

1. 레지스터



- 왼쪽의 그림은 32비트 기반인 MIPS에서 사용하는 레지스터 입니다.

- 왼쪽 그림의 레지스터는 총 32개의 공간으로 되어 있으며 각각의 공간은 32bits를 저장할 수 있습니다. 그러므로 2^{32} 가 넘는 메모리 주소를 레지스터는 읽을 수 없습니다.

- 32비트 컴퓨터의 레지스터는 각각의 공간이 저장할 수 있는 크기가 32bits 입니다.

- 64비트 컴퓨터의 레지스터는 각각의 공간이 저장할 수 있는 크기가 32bits인 놈과 64bits 인 놈이 섞여 있습니다.

- 32비트 컴퓨터에서 long long type을 사용할 때 2개의 레지스터를 이용합니다.

2. RISC & SISC

1) RISC (Reduced Instruction Set Computer)

= 명령어의 개수를 최소화하여 단순하게 제작된 CPU

- 연산할 때 메모리에 있는 값은 전혀 사용하지 않고 오로지 레지스터에 있는 값만 사용합니다.
- RISC를 기반으로 보면 컴퓨터는 연산을 하려면 레지스터로 데이터를 Load 해야 합니다.
- 메모리에 있는 배열의 값 A[8]을 레지스터로 가져오려면 , 배열 A의 시작주소에 가서 8칸 떨어져 있는 A[8]을 가져와야 합니다.

ex) AMD

2) CISC (Complex Instruction Set Computer)

= 복잡하고 기능이 많은 명령어로 구성된 CPU

- 연산할 때 레지스터와 레지스터 연산 , 레지스터와 메모리 연산 , 메모리와 메모리 연산을 모두 사용합니다.

ex) Intel

3. 메모리와 주소

1) Word Addressing

= 4 바이트 단위로 데이터를 저장합니다.

- 1바이트인 char 2바이트인 short를 사용할 때 어렵기 때문에 현대에는 사용하지 않습니다.

2) Byte Addressing

= 1 바이트 단위로 데이터를 저장합니다.

- 현대의 대부분의 메모리는 Byte Addressing을 이용합니다.

3) Alignment Restriction

= '4 바이트 데이터는 4의 배수 번지부터 사용해야 한다.'라는 제약을 갖는 정책입니다. ex) 0번지 , 4번지 , 8 번지

- 대부분의 컴퓨터가 이 정책을 따릅니다.

- 과거의 pentium 컴퓨터는 Alignment Restriction을 지키지 않았습니다. 4바이트의 데이터도 2번지나 13번지 같이 4의 배수가 아닌 번지부터 데이터를 저장하였습니다. 메모리 공간의 낭비를 줄일 수 있으나 , 메모리 구조상 해당 데이터를 읽는 게 매우 어려워져 , 성능이 매우 떨어집니다. 예를 들어 2번지 ~ 5번지를 차지하는 데이터는 레지스터에 올리기 위해 메모리에서 읽으려면 0번지 ~ 3번지로 저장 된 데이터 보다 2배의 clock cycle을 사용하게 됩니다. 그 당시에는 메모리가 비쌌기 때문에 공간의 낭비를 줄이는 것이 중요했었습니다.

- Alignment Restriction을 지켜야 파이프라인에 유리합니다.



4) Endianness

= Int형 데이터 A가 메모리의 100번지 ~ 103번지에 저장되어 있습니다. 레지스터에 A를 올릴 때 100번지를 A의 주소를 할 것 인지 103번지를 A의 주소로 할 것인지를 정하는 문제입니다. 다시 말해 , 레지스터에 100 101 102 103 으로 올라갈 것인지 103 102 101 100 으로 올라갈 것인지에 대한 내용입니다.

= Little Endian vs Big Endian

CPU가 메모리에 데이터를 저장할 때 어느 순서로 저장하는가에 따라서 리틀엔디안과 빅엔디안으로 나뉘게 됩니다.

리틀엔디안(Little Endian)은 메모리의 첫 주소에 하위 데이터(데이터의 맨 오른쪽)부터 저장하고 빅 엔디안(Big Endian)은 메모리의 첫 주소에 상위 데이터(데이터의 맨 왼쪽)부터 저장합니다.

예를들어 `int a = 0x12345678` 을 저장한다고 할 때 아래와 같이 저장됩니다.

타입	메모리 (int는 4byte 이므로 1byte씩 저장)
리틀엔디안	0x78 0x56 0x34 0x12
빅엔디안	0x12 0x34 0x56 0x78

4. 변수와 상수의 속도

- c++에서 항상 그런 건 아니지만 대개 상수가 변수보다 아마 빠를 것이라고 전문가들은 대답합니다. 현대의 컴퓨터 구조가 너무 다양하기 때문에 실제 benchmark에서 이 차이를 확인하는 것은 어렵습니다.

<3강_컴퓨터 연산>

1. 오버플로우

```
int thirty_one = 0x7fffffff;
cout << "31비트가 1 : " << thirty_one << "\n";
int thirty_two = 0x8fffffff;
cout << "32비트가 1 : " << thirty_two << "\n";
int bosu = 0x70000000;
cout << "thirty_two의 2의 보수 : " << bosu+1 << "\n";
```

```
pjw96@ParkJiWon ~/BOJ/source
$ ./a.exe
31비트가 1 : 2147483647
32비트가 1 : -1879048193
thirty_two의 2의 보수 : 1879048193
```

- thirty_one은 31비트가 모두 1이고 맨 앞 비트가 0이므로 int의 최댓값이 나옵니다.
- thirty_two는 32번째 비트가 1이므로 음수가 나옵니다. 오버플로우를 발생시키지 않으려면 unsigned int OR long long을 사용해야 합니다.
- thirty_two가 -1879048193이 나오는 이유는 맨 앞 비트가 1이므로 2의 보수를 취했기 때문입니다. 맨 앞의 16진수가 8(hex)이므로 1000(2)을 뒤집어서 0111(2)로 만들면 7(hex)로 되고 , 나머지 하위 16진수는 F(hex)이므로 0(hex)으로 바뀝니다. 그 결과 변수 'bosu+1' 값과 동일해집니다.
- C_Language는 오버플로우를 무시하고 음수로 나타내버립니다. 그러므로 원하지 않는 값으로 지정되는 것을 찾기가 어렵습니다.

2. 나눗셈

1) Division by zero

= C_Language에서 제수(divisor)가 0임을 컴퓨터가 발견하면 exception으로 처리하고 Warning을 나타냅니다.

2) 부호 있는 나눗셈과 나머지 연산(Moduler)

- 나눗셈은 사람과 동등하게 $-4 / 2 = -2$ 로 잘 처리 합니다.
- 모듈러 연산은 음수가 포함되면 값이 이상하게 나오므로 $(n+4)\%3$ 처럼 피제수를 양수화 시킵니다.

3. 컴퓨터는 어떻게 소수를 나타낼까요?

= Floating Point (=부동 소수점)로 나타냅니다.

浮動 부동 

붙박여 있지 않고 떠서 움직임. 진득하지 못하고 들뜸

- '부'는 아닐 부가 아닙니다.

<Floating Point를 나타내는 세계 공통 규칙>

Floating point numbers in binary form

$$\pm 1.xxxxxxxxx_{\text{two}} \times 2^{yyyy}$$

① sign

= 부호를 나타내며 값이 1로 고정입니다.

- 1비트

② Fraction

= 소수점 이하의 수를 나타내며 규칙에서 x로 나타냅니다.

- 52비트

③ Exponent

= 지수를 나타내며 규칙에서 y로 나타냅니다.

- 11비트

- 1 + 52 + 11 = 64비트로 double 타입을 나타냅니다.

<5강_메모리 계층 구조>

1. Memory Hierarchy

1) Overview

- 계층화의 목표 : Minimum cost -> Maximum performance
- 메모리(DRAM)는 CPU에 비해 매우 느리므로 둘의 차이를 극복하기 위해 캐시메모리(SRAM)가 존재합니다.
- CPU만 빨라봐야 의미가 없고 , 메모리와 캐시메모리도 어느 정도 빨라야 컴퓨터 성능이 좋아집니다.
- Illusion을 통해 빠르고 많은 메모리가 있는 것처럼 보이게 한다.
- CPU와 가까이 있는 메모리를 상위 계층 메모리(캐시 메모리)라 하고 멀리 있는 메모리를 하위 계층 메모리(RAM)라 합니다.
- 캐시메모리는 CPU에 들어 있습니다.

2) Principle of Locality(지역성의 원칙)

- 캐시메모리는 매우 작은 용량이지만 왜 유용한지에 관한 질문에 대한 해답이 될 수 있습니다.
- 컴퓨터는 의외로 메모리의 일부분만 사용합니다.

① Temporal locality(시간적 지역성)

- = 최근에 사용한 메모리의 주소 공간은 다시 사용될 가능성이 매우 큼.
- 프로그램들 반복문을 많이 사용하기 때문입니다.

② Spatial locality(공간적 지역성)

- = 100번지의 메모리를 읽으면 96번지나 104번지를 읽을 가능성이 높습니다.
- 프로그램이 배열 , 자료구조를 많이 사용하기 때문입니다.

3) 지역성의 원칙을 따르는 Cache-Friendly Code를 작성하여 Cache Hit를 높이는 효과적인 코딩을 합시다.

① C++ Vector를 사용합시다.

Use appropriate `c++` containers

A simple example of cache-friendly versus cache-unfriendly is `c++` 's `std::vector` versus `std::list`. Elements of a `std::vector` are stored in contiguous memory, and as such accessing them is *much* more cache-friendly than accessing elements in a `std::list`, which stores its content all over the place. This is due to spatial locality.

A very nice illustration of this is given by Bjarne Stroustrup in [this youtube clip](#) (thanks to @Mohammad Ali Baydoun for the link!).

② 가상 함수를 사용하지 맙시다.

In the context of `c++`, `virtual` methods represent a controversial issue with regard to cache misses (a general consensus exists that they should be avoided when possible in terms of performance). Virtual functions can induce cache misses during look up, but this only happens *if* the specific function is not called often (otherwise it would likely be cached), so this is regarded as a non-issue by some. For reference about this issue, check out: [What is the performance cost of having a virtual method in a C++ class?](#)

참고 문헌 : <https://stackoverflow.com/questions/16699247/what-is-a-cache-friendly-code/16699282#16699282>

4) 추가 용어

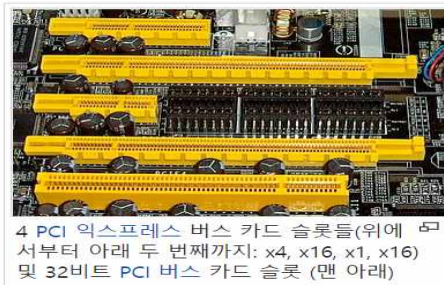
- Block : CPU와 캐시는 워드 단위로 데이터를 교환하지만 , 캐시와 메모리는 n개의 워드로 이루어진 블록 단위로 데이터를 교환합니다.
- Hit : CPU가 원하는 데이터가 캐시메모리에 존재할 때.
- Miss : CPU가 원하는 데이터가 캐시메모리에 존재할 때.
- Hit time : 캐시메모리에 접근하고 , 여기에 원하는 데이터의 유무를 체크하는데 걸리는 시간.
- Miss penalty : 캐시에 없어서 메모리에 접근하고 메모리에서 찾은 블록을 캐시의 블록과 교체하는 시간 + 그 블록을 CPU에 전달하는 시간
- Miss penalty가 Hit time 보다 훨씬 큼니다.
- **Average memory access time = Hit time + Miss rate X Miss penalty**

<버스>

- 버스는 컴퓨터 안의 부품들 간에, 또는 컴퓨터 간에 데이터와 정보를 전송하는 통로(통신 시스템)이다. 이러한 표현에는 관련된 모든 하드웨어 부품들 (선, 광 파이버 등) 및 통신 프로토콜을 포함한 소프트웨어를 아우른다.

초기의 컴퓨터 버스들은 여러 연결들을 갖춘 병렬의 전기적 선들이었으나, 현재 이 용어는 병렬 전기적 버스와 동일한 논리적 기능을 제공하는 물리적인 배치에 사용된다. 현대의 컴퓨터 버스들은 병렬 및 비트 시리얼 연결을 둘 다 지원하며, 이들 모두 멀티드롭 (전기적 병렬) 또는 데이지 체인 토폴로지로 선을 구성하거나 USB처럼 스위치 허브에 의해 연결할 수도 있다. 입출력 장치와 연결된 통로는 입출력 버스(I/O bus), 메모리와 다른 장치를 이어주는 통로는 메모리 버스(memory bus)라고 한다.

- **하드웨어들을 연결한 전기적 통로(선)**
- 버스도 하드웨어입니다.



2. 캐시 메모리

1) 정의

- 레지스터와 메모리 사이의 속도 차이를 줄이기 위해 레지스터에서 사용할 확률이 높은 데이터를 레지스터와 메모리 사이에 있는 캐시 메모리에 저장해서 속도를 향상시킵니다.
- 1 클럭 안에 캐시 메모리에서 데이터를 찾아야 하므로 캐시 메모리 내에서는 Search Algorithm을 이용하지 않습니다.

2) 용어

① valid

= 캐시 메모리의 블록 안에 어떤 데이터가 저장되어 있는지는 알 수 있으나 데이터가 저장 되어 있다면 1 , 데이터가 저장 되어 있지 않으면 0으로 나타내는 1비트의 flag

② Index (Cache Address)

= 32비트 레지스터 OR 32비트 메모리에서 사용하는 32비트 주소의 일부분을 떼어 내서 캐시에서 사용하는 주소로 만듭니다.

- 보통 하위 n비트를 사용하며 , int를 많이 쓰는 구조상 최하위 2비트(0번 비트 , 1번 비트)는 무시합니다.

ex) 32비트 주소 중에서 하위 10비트를 사용하면 0번 비트 , 1번 비트를 무시한 '2번 비트 ~ 11번 비트'를 주소로 이용합니다.

③ Tag

= 32비트 주소 중에서 Index를 제외한 주소

- valid에 저장된 값이 1이고 데이터가 존재할 때 Tag값을 비교해서 같으면 Hit , 같지 않으면 Miss가 됩니다.

- Hit가 되면 캐시메모리에 저장된 Data를 레지스터에서 읽습니다.

ex) 32비트 주소 중에서 하위 10비트를 Index로 사용하고 0번 비트 , 1번 비트는 제외하는 워드 주소를 사용할 때 12번 비트 ~ 31번 비트를 Tag로 사용합니다.

④ Data

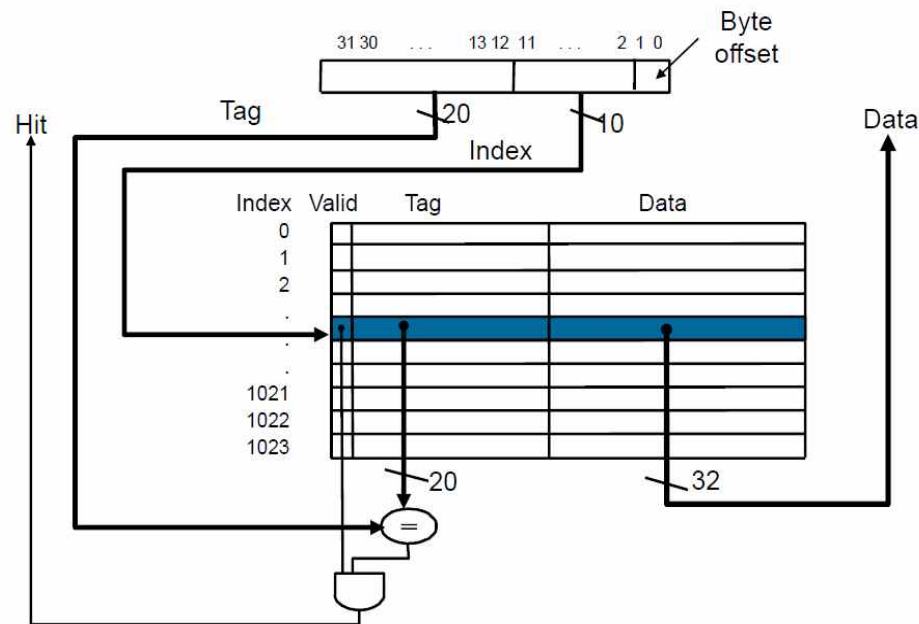
= 메모리의 32비트 주소

- 캐시메모리에도 당연히 메모리의 원형 데이터를 저장해야 메모리에 접근하지 않습니다.

⑤ 캐시메모리의 크기

- 캐시메모리는 다수의 캐시 블록으로 이루어져 있고 각각의 캐시 블록은 valid, tag, data로 구성되어 있습니다.
- 하지만 보통 data 블록의 크기를 캐시 블록의 크기로 나타내기 때문에, 캐시 메모리의 크기는 data 블록의 크기 * 캐시메모리에 존재하는 캐시 블록의 개수로 나타냅니다.

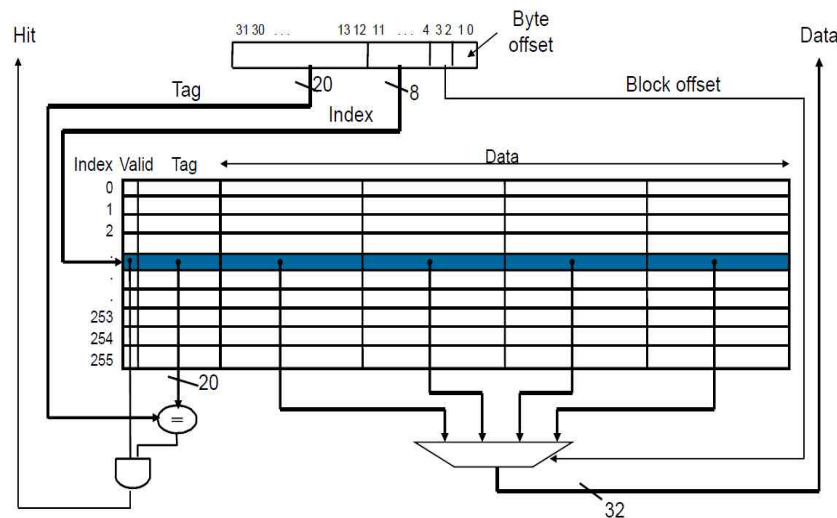
3) 4KB 캐시메모리에 data 블록의 크기가 1word(=4byte)인 경우 : 매우 비효율적인 방법



- valid는 캐시 블록에 데이터의 유무를 1비트로 저장합니다.
- Index는 메모리 주소의 11번 비트 ~ 2번 비트를 저장해서 10비트의 크기로 저장합니다.
- Tag는 메모리 주소의 31번 비트 ~ 12번 비트를 저장해서 20비트의 크기로 저장합니다.
- Data에는 원래의 메모리 주소 값을 온전히 저장하므로 32비트의 크기로 저장합니다.
- 4KB에 4byte의 데이터를 저장하므로 1024개의 캐시 블록이 존재합니다.
- 통상적으로 캐시 블록의 크기를 data 블록의 크기로 하여 그림에서는 캐시 블록의 크기가 4byte 이지만, 실제로는 Tag와 index의 크기까지 포함하므로 53비트(=1 + 20 + 32)가 캐시 블록의 크기입니다.
- 확실하지 않지만 그러므로 실제 블록의 크기가 통상적으로 생각한 블록의 크기인 32 bits 보다 약 1.5배 크므로 4KB 캐시 메모리에 1024개의 캐시 블록을 저장할 수 없을 것 입니다.

4) 4KB 캐시메모리에 data 블록의 크기가 4word(=16bytes)인 경우

- 2^2 words/block, cache size = 4 KB ($m=2, n=8$)



<Data Block의 크기가 이전 모델에 비해서 4배 증가>

- valid는 동일하게 1비트 flag 역할을 합니다.
- index는 data block의 크기가 4배 증가해서 4배 감소한 256개가 됩니다.
- tag는 동일합니다.
- 이전보다 4배가 증가했으므로 하위 4비트(3번 비트 ~ 0번 비트)를 모두 무시합니다.
- data block의 크기가 4배 증가해서 128bit가 됩니다. 하지만 레지스터에는 32bit 값 1개만 전달해야 하므로 하위 4개 비트 중 3번 비트와 2번 비트를 offset으로 하여 offset 값(0~3)을 이용하여 4개의 32bit 중 1개의 32bit를 선택하여 레지스터로 전달합니다.

5) Multi-Level Cache

- 캐시 계층을 3단계로(L1 Cache , L2 Cache , L3 Cache) 계층화 시켜서 캐시의 성능(hit/miss)을 향상 시킵니다.
- 각각의 계층에서 발생하는 miss를 local miss rate라고 하고 모든 계층을 합쳐서 발생하는 miss를 global miss rate라고 합니다. 당연히게도 global miss rate가 더 중요한 개념입니다.

Cache	
Cache L1:	64K (per core)
Cache L2:	256K (per core)
Cache L3:	16MB (shared)

- 왼쪽의 그림은 Intel의 i9_9900k의 3단계 캐시 메모리 입니다.

- K는 KB입니다.

① L1 Cache : CPU 가장 근처이므로 pipelining의 성능과 관련되어 있으므로 매우 빨라야 합니다.

② L2 , L3 Cache : 속도 보다는 miss-rate를 최대한 줄여서 Memory에 접근을 하지 않도록 합니다.

<6강_병렬 프로세서>

1. Multi-Core

- CPU는 Processor(프로세서)와 같은 뜻이며 , Core와는 포함관계 입니다.
- CPU 내부에는 1개 또는 여러 개의 Core(CPU의 핵심 작업을 수행하는 핵심 부품)가 들어 갈 수 있습니다.
- 과거에는 CPU에 1개의 코어만이 존재해서 구분하지 않아도 괜찮았으나 , 현대에는 최소 2개 이상의 코어로 CPU가 이루어져 있고 , CPU의 일처리 단위가 코어이므로 , CPU와 코어를 구분해서 사용해야 합니다. 예를 들어 , 6개의 CPU로 처리한다고 적힌 글은 1개의 CPU가 6개의 코어로 이루어져 있고 , 6개의 코어를 이용해서 처리한다고 해석해야 합니다. 그리고 교재에서도 코어를 사용하길 권장합니다.
- 코어는 1개의 프로그램 문맥(Process OR Thread)을 실행할 수 있습니다. 현대에는 Hyper-Threading 기술로 H/W의 지원을 받아 1개의 코어에서 복수의 프로그램 문맥을 실행할 수 있습니다.
- Single-Core 환경에서 2개의 thread를 수행하면 , 1번 thread -> 2번 thread -> 1번 thread -> 2번 thread를 빠르게 반복하여 수행합니다.
- Multi-Core 환경에서 2개의 thread를 수행하면 , 1번 코어가 1번 thread를 수행하고 , 2번 코어가 2번 thread를 수행합니다.
- 위의 내용은 다음 장의 'Concurrency(동시성) vs Parallelism(병렬성)'의 그림을 참조합니다.
- 동일한 CPU성능이라고 가정할 때 , Multi-Core 환경이라고 해서 Single-Core 보다 2 배의 성능을 내는 것은 아닙니다. 프로그램이 처리해야 하는 계산 작업 사이에서 여러 개의 코어가 맡아서 처리할 수 없어서 하나의 코어만 사용해야 하는 작업이 생기면 나머지 코어는 그 작업이 처리되는 동안 놀고먹는 모양새가 됩니다. 이렇게 프로그램 내부에서 역할 분담(병렬화)이 되지 않는 부분이 많아지면 코어를 아무리 많이 갖는 CPU라 할지라도 성능 향상에 한계가 생깁니다. 이를 전문 용어로 Amdahl's Law라고 합니다.
- Reference : <https://stackoverflow.com/questions/19225859/difference-between-core-and-processor>

<Multi-Core 종류>

명칭	코어 개수	특징
Dual-Core	2개	구시대의 유물
Quad-Core	4개	저가형
Hexa-Core	6개	2021년 기준 평균
Octa-Core	8개	2021년 기준 중고급
Deca-Core	10개	2021년 기준 고가 - Intel의 Xeon 시리즈 - 현재 64코어 128스레드의 Ryzen 3990X까지 존재하지만 다른 세계의 이야기

<Multi-Core의 현실>

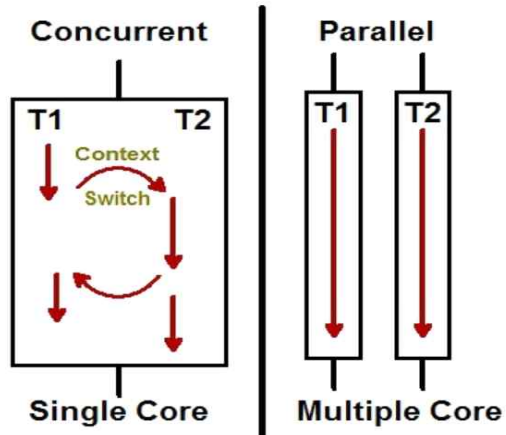


2. Multi-Thread

= 하나의 CPU에서 여러 개의 thread를 지원하는 기능입니다.

- 보통 하나의 코어에서 2개의 thread를 처리합니다. Quad-Core CPU에서 Multi_thread 방식으로 스케줄링을 하면 , 각각의 코어가 2개의 thread를 처리하므로 , 최대 8개의 thread를 처리합니다.
- PCB처럼 thread를 추상화한 구조체인 TCB(Thread Control Block)가 존재합니다.
- 하나의 프로세스를 쪼개서 독립적인 미니 프로세스인 Thread로 스케줄링을 운용합니다.
- 스케줄링의 단위가 Thread입니다.
- 1번 thread가 I/o해서 block 된다고 해도 같은 프로세스의 3번 thread는 block 되지 않을 수 있습니다. -> CPU Utilization 극대화
- 문맥 교환의 단위가 Thread이므로 process scheduling보다 경제적입니다.
- 프로세스의 병렬성을 높여 Multi-Processor 활용이 가능합니다.
- 게임 프로그래밍에서 여러 캐릭터들을 동시에 움직이는 기법이 Multi-Thread 방식이며 , 각각의 캐릭터가 하나의 Thread 입니다.
- Multi-Core는 "하드웨어 관점"에서의 물리적 구성단위이고 , Multi-Thread는 "소프트웨어 관점"에서의 논리적 작업 처리 단위라는 차이점이 있다.

3. Concurrency(동시성) vs Parallelism(병렬성)



동시성	병렬성
동시에 실행되는 것 같이 보이는 것	실제로 동시에 여러 작업이 처리되는 것
싱글 코어에서 멀티 쓰레드(Multi thread)를 동작시키는 방식	멀티 코어에서 멀티 쓰레드(Multi thread)를 동작시키는 방식
한번에 많은 것을 처리	한번에 많은 일을 처리
논리적인 개념	물리적인 개념

- 좌측 그림은 Thread를 2개 처리하는 예제입니다.

① Single Core

= Thread 단위로 scheduling하여 빠르고 자주 Context Switching을 하여 CPU 제어 권한을 넘겨주어 처리합니다.

② Multiple Core

= Thread_1은 1번 Core가 , Thread_2는 2번 Core가 처리하여 동시에 독립적으로 처리합니다.

*** OS 강의에서 대단원_2가 Concurrency이므로 , 이 강의에서 배우는 Multi-Thread는 싱글 코어에서 동작하는 방식입니다. ***

<reference>

<https://seamless.tistory.com/42>

https://m.blog.naver.com/PostView.nhn?blogId=and_lamyland&logNo=221182893855&proxyReferer=https:%2F%2Fwww.google.com%2F

4. Coroutine(코루틴) VS Thread(스레드)

1) 동기식 처리 모델 vs 비동기식 처리 모델

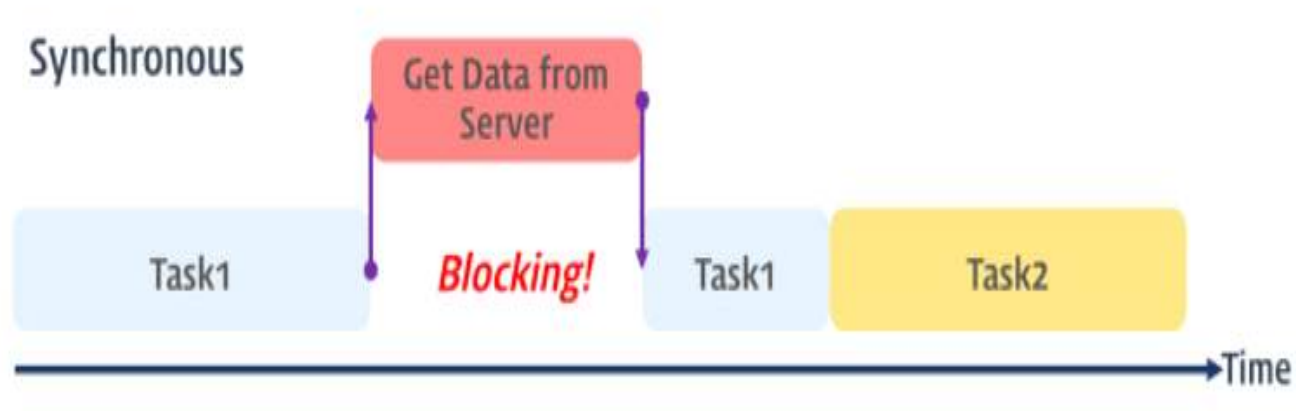


(a) 동기



(b) 비동기

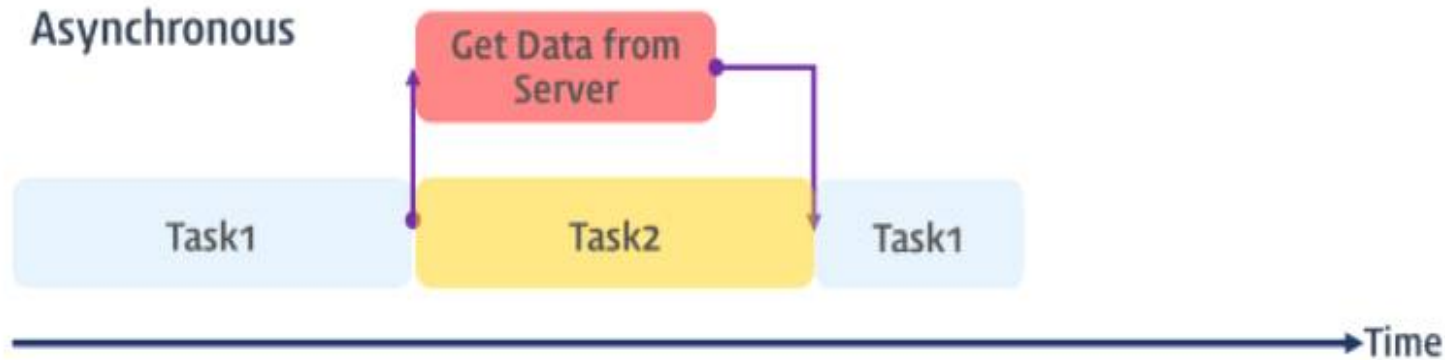
① 동기식(Synchronous) 처리 모델 : 직렬(=순차) 처리



동기식 처리 모델(Synchronous processing model)

- 직렬적으로 태스크(task)를 수행합니다. 태스크는 순차적으로 실행되며 어떤 작업이 수행 중이면 다음 작업은 대기하게 됩니다.
 - 서브루틴 간에 **명확한 실행 순서가 존재**하기 때문에 작업이 A,B,C 순으로 실행되기를 기대한다면 반드시 A->B->C의 순서로 실행되어야 합니다. 그러므로 B는 A가 실행이 완료되기를 기다리며 , C는 A와 B가 실행 완료되기를 기다립니다.
 - **Coroutine을 동기식 처리 모델이라고 확실히 말할 수는 없으나 적어도 비동기식 처리 모델은 절대 아니며** , 명확한 실행 순서가 존재합니다.
- ex) 서버에서 데이터를 가져와서 화면에 표시하는 작업을 수행할 때, 서버에 데이터를 요청하고 데이터가 응답될 때까지 이후 태스크들은 블로킹(blocking, 작업 중단)됩니다.

② 비동기식(Asynchronous) 처리 모델 : 병렬(=동시) 처리



비동기식 처리 모델(Asynchronous processing model)

- 병렬(Parallel)적으로 태스크(task)를 수행합니다. 태스크가 종료되지 않은 상태라 하더라도 대기하지 않고 다음 태스크를 실행합니다.
- 서브루틴 간에 명확한 실행 순서가 존재하지 않기 때문에 A->B->C의 순서로 실행될 수도 있고, B->C->A의 순서로 실행될 수도 있습니다. 따라서 프로그래밍 시에 반드시 이점을 고려해야 합니다. 이러한 경우에는 서브루틴이 완료되었다는 것을 전달하기 위해 callback 패턴을 사용하기도 합니다.
- 대표적인 비동기식 처리 모델로는 Thread Programming과 Firebase가 있습니다.

This means that any Firebase API that needs to deal with data on disk or network is implemented in an *asynchronous* style.

ex) 서버에서 데이터를 가져와서 화면에 표시하는 태스크를 수행할 때, 서버에 데이터를 요청한 이후 서버로부터 데이터가 응답될 때까지 대기하지 않고(Non-Blocking) 즉시 다음 태스크를 수행한다. 이후 서버로부터 데이터가 응답되면 이벤트가 발생하고 이벤트 핸들러가 데이터를 가지고 수행할 태스크를 계속해 수행합니다.

2) Coroutine

- = 특정 작업을 시간 혹은 다른 작업의 처리가 완료되었을 때 단계적으로 진행 시키는 함수 입니다. 다시 말해 , 다른 작업이 완료될 때까지 현재 작업을 기다리는 루틴을 사용할 수 있습니다. 결론적으로 작업 처리의 순서(Scheduling)를 개발자가 제어할 수 있습니다.
- Coroutine은 Thread가 아닙니다.
- Coroutine은 비동기가 아니기 때문에 작업들이 동시에 처리되지 않습니다.
- 매 Frame 마다 Update 구문을 확인하지 않기 때문에 프로그램의 성능 향상에 큰 영향을 미칩니다.
- 코드의 가독성이 좋아집니다.

3) Thread

- = 한 프로세스 내에 존재하는 실행 흐름.
- 프로세스를 쪼갠 미니 프로세스라고 이해했습니다.
- Thread는 비동기적으로 처리하기 때문에 다른 Thread와 함께 각각의 작업을 동시에 처리 할 수 있습니다.
- 현대 CPU는 다중 코어로 발전해서 OS는 자원들을 충분히 활용하기 위해 병렬 프로그래밍을 하게 되는데 , 병렬 처리의 핵심도구가 Thread 입니다.
- 코드가 복잡하고 , 실행 순서를 파악하기 어렵습니다.
- 완벽해보이지만 Thread 간의 자원 경쟁(상호 배제)과 데드락이라는 문제점이 존재합니다.

4) Reference

- <https://3dmpengines.tistory.com/2035>
- <https://silisian.tistory.com/44>
- <https://showx123.tistory.com/62>
- 동시성(Concurrency) vs 병렬성(Parallelism) : <https://seamless.tistory.com/42>

5. Multi-Core Programming이 어려운 이유

- Multi-Core 환경에서 수행되는 S/W를 만드는 것은 어렵기 때문에 이런 유형의 응용 프로그램이 적습니다.
- 어떤 프로세스의 작업량이 100일 때 4개의 스레드가 각각 25씩의 작업량이면 , Quad-Core환경에서 매우 빠르게 수행되지만 , 스레드가 97,1,1,1로 나누어져 있다면 성능이 좋지 않을 것 입니다. 즉 , Multi-Core 환경에서 최적의 성능을 뽑아내려면 스레드가 동일한 크기의 작업량을 분배 받아야 합니다.
- Amdahl's Law에 따르면 Multi-Core Programming에서의 속도 개선에는 어려움이 존재합니다.